

Язык программирования C++

Лекция 16: STL, функциональные объекты, стандартные алгоритмы, iterator traits, категории итераторов, tag dispatching

Никита Лисица

2026

Напоминание: итераторы

- Классы, позволяющие итерироваться по коллекции (в том числе указывать на конкретный её элемент)

Напоминание: итераторы

- Классы, позволяющие итерироваться по коллекции (в том числе указывать на конкретный её элемент)
- Позволяют писать универсальный код, не зависящий от конкретного контейнера

Напоминание: итераторы

- Классы, позволяющие итерироваться по коллекции (в том числе указывать на конкретный её элемент)
- Позволяют писать универсальный код, не зависящий от конкретного контейнера
- По семантике и синтаксису имитируют указатели:

```
std::list<int> l;  
...  
auto it = l.begin();  
// Сдвинуться на следующий элемент  
++it;  
// Прочитать текущий элемент  
int x = *it;  
// Записать текущий элемент  
*it = 42;
```

Напоминание: итераторы

- Диапазон элементов задаётся парой в виде полуоткрытого интервала `[begin, end)`

Напоминание: итераторы

- Диапазон элементов задаётся парой в виде полуоткрытого интервала `[begin, end)`
- `begin` – итератор на первый элемент диапазона

Напоминание: итераторы

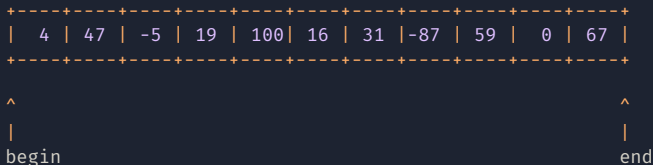
- Диапазон элементов задаётся парой в виде полуоткрытого интервала `[begin, end)`
- `begin` – итератор на первый элемент диапазона
- `end` – итератор за последний элемент диапазона (на первый элемент *после* диапазона)

Напоминание: итераторы

- Диапазон элементов задаётся парой в виде полуоткрытого интервала `[begin, end)`
- `begin` – итератор на первый элемент диапазона
- `end` – итератор за последний элемент диапазона (на первый элемент *после* диапазона)

Напоминание: итераторы

- Диапазон элементов задаётся парой в виде полуоткрытого интервала `[begin, end)`
- `begin` – итератор на первый элемент диапазона
- `end` – итератор за последний элемент диапазона (на первый элемент *после* диапазона)



- `end` часто используется для указания отсутствия элемента (напр. при поиске)

- Класс, у которого перегружен оператор вызова `operator()`

```
struct add {  
    int operator()(int x, int y) const {  
        return x + y;  
    }  
};  
  
add func;  
int x = func(3, 4); // x = 7
```

- Класс, у которого перегружен оператор вызова `operator()`

```
struct add {  
    int operator()(int x, int y) const {  
        return x + y;  
    }  
};  
  
add func;  
int x = func(3, 4); // x = 7
```

- **NB:** Раньше такие классы называли *функторами*, но это пересекается с понятием из теории категорий, поэтому сейчас лучше говорить *функциональный объект*

- По смыслу похожи на функции, но мощнее

- По смыслу похожи на функции, но мощнее: позволяют хранить в себе данные

```
struct less_than {  
    int reference;  
  
    bool operator()(int x) const {  
        return x < reference;  
    }  
};  
  
less_than pred(10);  
pred(8); // true  
pred(12); // false
```

Предикаты

- Функциональные объекты, возвращающие `bool`, называют *предикатами*

Предикаты

- Функциональные объекты, возвращающие **bool**, называют *предикатами*
- Предикат с одним аргументом – *унарный (unary predicate)*

Предикаты

- Функциональные объекты, возвращающие **bool**, называют *предикатами*
- Предикат с одним аргументом – *унарный (unary predicate)*
- Предикат с двумя аргументами – *бинарный (binary predicate)*

Предикаты

- Функциональные объекты, возвращающие **bool**, называют *предикатами*
- Предикат с одним аргументом – *унарный (unary predicate)*
- Предикат с двумя аргументами – *бинарный (binary predicate)*

```
// Бинарный предикат
struct contains {
    int reference;

    bool operator()(int x, int y) const {
        return x <= reference && reference <= y;
    }
};

contains pred(10);
pred(6, 7); // false
pred(8, 12); // true
pred(12, 15); // false
```

- Функциональные объекты могут менять своё состояние при вызове

- Функциональные объекты могут менять своё состояние при вызове

```
struct accumulator {  
    int sum = 0;  
  
    void operator()(int x) {  
        sum += x;  
    }  
};  
  
accumulator f;  
f(10);  
f(25);  
f(32);  
  
// f.sum == 67
```

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности
 - Алгоритмы разбиения и сортировки

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности
 - Алгоритмы разбиения и сортировки
 - Алгоритмы на сортированных последовательностях

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности
 - Алгоритмы разбиения и сортировки
 - Алгоритмы на сортированных последовательностях
 - Алгоритмы работы с кучей (структурой данных)

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности
 - Алгоритмы разбиения и сортировки
 - Алгоритмы на сортированных последовательностях
 - Алгоритмы работы с кучей (структурой данных)
 - Алгоритмы генерации перестановок

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности
 - Алгоритмы разбиения и сортировки
 - Алгоритмы на сортированных последовательностях
 - Алгоритмы работы с кучей (структурой данных)
 - Алгоритмы генерации перестановок
 - И т.д.

- В STL более 100 алгоритмов, почти все – в заголовочном файле `<algorithm>`
 - Алгоритмы поиска
 - Алгоритмы, модифицирующие последовательности
 - Алгоритмы разбиения и сортировки
 - Алгоритмы на сортированных последовательностях
 - Алгоритмы работы с кучей (структурой данных)
 - Алгоритмы генерации перестановок
 - И т.д.
- Мы рассмотрим только часть из них

- `swap(T& x, T& y)` – поменять местами два значения

- `swap(T& x, T& y)` – поменять местами два значения
- `iter_swap(It p, It q)` – поменять местами значения, на которые указывают итераторы

- `swap(T& x, T& y)` – поменять местами два значения
- `iter_swap(It p, It q)` – поменять местами значения, на которые указывают итераторы
- `min(const T& x, const T& y)` – минимум из двух значений

- `swap(T& x, T& y)` – поменять местами два значения
- `iter_swap(It p, It q)` – поменять местами значения, на которые указывают итераторы
- `min(const T& x, const T& y)` – минимум из двух значений
- `max(const T& x, const T& y)` – максимум из двух значений

- `swap(T& x, T& y)` – поменять местами два значения
- `iter_swap(It p, It q)` – поменять местами значения, на которые указывают итераторы
- `min(const T& x, const T& y)` – минимум из двух значений
- `max(const T& x, const T& y)` – максимум из двух значений
- `clamp(const T& x, const T& min, const T& max)` – ограничить значение диапазоном `[min, max]`

Min/max с предикатом

- Всегда, когда стандартный алгоритм использует сравнение оператором $<$, у него есть версия, принимающая произвольный бинарный предикат

Min/max с предикатом

- Всегда, когда стандартный алгоритм использует сравнение оператором `<`, у него есть версия, принимающая произвольный бинарный предикат

```
struct point {  
    float x;  
    float y;  
};  
  
struct less_by_x {  
    bool operator()(const point& p1, const point& p2) const {  
        return p1.x < p2.x;  
    }  
};  
  
point p1{1.f, 4.f};  
point p2{3.f, 2.f};  
  
// Шаблонные параметры выводятся автоматически  
std::max(p1, p2, less_by_x());
```

Немодифицирующие алгоритмы на последовательностях

- `count(It begin, It end, const T& x)` – сколько раз в последовательности встречается значение `x`

Немодифицирующие алгоритмы на последовательностях

- `count(It begin, It end, const T& x)` – сколько раз в последовательности встречается значение `x`
- `count_if(It begin, It end, UnaryPred pred)` – сколько раз в последовательности встречаются элементы, удовлетворяющие предикату

Немодифицирующие алгоритмы на последовательностях

- `count(It begin, It end, const T& x)` – сколько раз в последовательности встречается значение `x`
- `count_if(It begin, It end, UnaryPred pred)` – сколько раз в последовательности встречаются элементы, удовлетворяющие предикату

```
struct divisible_by {  
    int divisor;  
    bool operator()(int x) const {  
        return (x % divisor) == 0;  
    }  
};  
  
std::vector<int> v;  
...  
std::count_if(v.begin(), v.end(), divisible_by(8));
```

- `all_of(It begin, It end, UnaryPred pred)` – правда ли, что все элементы последовательности удовлетворяют предикату

- `all_of(It begin, It end, UnaryPred pred)` – правда ли, что все элементы последовательности удовлетворяют предикату
- `any_of(It begin, It end, UnaryPred pred)` – правда ли, что хотя бы один элемент последовательности удовлетворяет предикату

Немодифицирующие алгоритмы

- `all_of(It begin, It end, UnaryPred pred)` – правда ли, что все элементы последовательности удовлетворяют предикату
- `any_of(It begin, It end, UnaryPred pred)` – правда ли, что хотя бы один элемент последовательности удовлетворяет предикату
- `none_of(It begin, It end, UnaryPred pred)` – правда ли, что все элементы последовательности не удовлетворяют предикату

- `equal(It1 begin1, It1 end1, It2 begin2)` – совпадают ли последовательности (должны быть одинаковой длины, поэтому не принимает `end2`)

- `equal(It1 begin1, It1 end1, It2 begin2)` – совпадают ли последовательности (должны быть одинаковой длины, поэтому не принимает `end2`)
- `equal(It1 begin1, It1 end1, It2 begin2, It2 end2)` – то же самое (последовательности разной длины считаются не совпадающими)

- `equal(It1 begin1, It1 end1, It2 begin2)` – совпадают ли последовательности (должны быть одинаковой длины, поэтому не принимает `end2`)
- `equal(It1 begin1, It1 end1, It2 begin2, It2 end2)` – то же самое (последовательности разной длины считаются не совпадающими)
- `pair<It1, It2> mismatch(It1 begin1, It1 end1, It2 begin2)` – вернуть первое несовпадение в последовательностях

- `equal(It1 begin1, It1 end1, It2 begin2)` – совпадают ли последовательности (должны быть одинаковой длины, поэтому не принимает `end2`)
- `equal(It1 begin1, It1 end1, It2 begin2, It2 end2)` – то же самое (последовательности разной длины считаются не совпадающими)
- `pair<It1, It2> mismatch(It1 begin1, It1 end1, It2 begin2)` – вернуть первое несовпадение в последовательностях
- `pair<It1, It2> mismatch(It1, It1, It2, It2)` – аналогично

- `equal(It1 begin1, It1 end1, It2 begin2)` – совпадают ли последовательности (должны быть одинаковой длины, поэтому не принимает `end2`)
- `equal(It1 begin1, It1 end1, It2 begin2, It2 end2)` – то же самое (последовательности разной длины считаются не совпадающими)
- `pair<It1, It2> mismatch(It1 begin1, It1 end1, It2 begin2)` – вернуть первое несовпадение в последовательностях
- `pair<It1, It2> mismatch(It1, It1, It2, It2)` – аналогично
- `lexicographical_compare(It1, It1, It2, It2)` – сравнить две последовательности лексикографически

- `It find(It begin, It end, const T& x)` – найти первое вхождение элемента `x` (или `end`, если такого нет)

- `It find(It begin, It end, const T& x)` – найти первое вхождение элемента `x` (или `end`, если такого нет)
- `It find_if(It begin, It end, UnaryPred pred)` – найти первое вхождение элемента, удовлетворяющего предикату

- `It find(It begin, It end, const T& x)` – найти первое вхождение элемента `x` (или `end`, если такого нет)
- `It find_if(It begin, It end, UnaryPred pred)` – найти первое вхождение элемента, удовлетворяющего предикату
- `It min_element(It begin, It end)` – найти минимальный элемент последовательности

- `It find(It begin, It end, const T& x)` – найти первое вхождение элемента `x` (или `end`, если такого нет)
- `It find_if(It begin, It end, UnaryPred pred)` – найти первое вхождение элемента, удовлетворяющего предикату
- `It min_element(It begin, It end)` – найти минимальный элемент последовательности
- `It max_element(It begin, It end)` – найти максимальный элемент последовательности

- `It find(It begin, It end, const T& x)` – найти первое вхождение элемента `x` (или `end`, если такого нет)
- `It find_if(It begin, It end, UnaryPred pred)` – найти первое вхождение элемента, удовлетворяющего предикату
- `It min_element(It begin, It end)` – найти минимальный элемент последовательности
- `It max_element(It begin, It end)` – найти максимальный элемент последовательности
- `pair<It, It> minmax_element(It begin, It end)` – найти минимальный и максимальный элементы последовательности (за один проход)

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`
- `generate(It begin, It end, Generator gen)` – заполнить последовательность результатами вызова `gen()`

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`
- `generate(It begin, It end, Generator gen)` – заполнить последовательность результатами вызова `gen()`
- `copy(ItIn src_begin, ItIn src_end, ItOut dst_begin)` – скопировать последовательность в последовательность, начинающуюся в `dst_begin`

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`
- `generate(It begin, It end, Generator gen)` – заполнить последовательность результатами вызова `gen()`
- `copy(ItIn src_begin, ItIn src_end, ItOut dst_begin)` – скопировать последовательность в последовательность, начинающуюся в `dst_begin`
- `copy_if(ItIn, ItIn, ItOut, UnaryPred)` – скопировать элементы, удовлетворяющие предикату

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`
- `generate(It begin, It end, Generator gen)` – заполнить последовательность результатами вызова `gen()`
- `copy(ItIn src_begin, ItIn src_end, ItOut dst_begin)` – скопировать последовательность в последовательность, начинающуюся в `dst_begin`
- `copy_if(ItIn, ItIn, ItOut, UnaryPred)` – скопировать элементы, удовлетворяющие предикату
- `reverse(It begin, It end)` – обратить порядок элементов последовательности

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`
- `generate(It begin, It end, Generator gen)` – заполнить последовательность результатами вызова `gen()`
- `copy(ItIn src_begin, ItIn src_end, ItOut dst_begin)` – скопировать последовательность в последовательность, начинающуюся в `dst_begin`
- `copy_if(ItIn, ItIn, ItOut, UnaryPred)` – скопировать элементы, удовлетворяющие предикату
- `reverse(It begin, It end)` – обратить порядок элементов последовательности
- `rotate(It begin, It middle, It end)` – циклически сдвинуть элементы последовательности

Модифицирующие алгоритмы

- `fill(It begin, It end, const T& x)` – заполнить последовательность копиями значения `x`
- `generate(It begin, It end, Generator gen)` – заполнить последовательность результатами вызова `gen()`
- `copy(ItIn src_begin, ItIn src_end, ItOut dst_begin)` – скопировать последовательность в последовательность, начинающуюся в `dst_begin`
- `copy_if(ItIn, ItIn, ItOut, UnaryPred)` – скопировать элементы, удовлетворяющие предикату
- `reverse(It begin, It end)` – обратить порядок элементов последовательности
- `rotate(It begin, It middle, It end)` – циклически сдвинуть элементы последовательности
- `transform(ItIn, ItIn, ItOut, UnaryOp op)` – как `copy`, но копируется результат применения `op(x)` к элементам

- `sort(It begin, It end)` – отсортировать последовательность

- `sort(It begin, It end)` – отсортировать последовательность
- `stable_sort(It begin, It end)` – отсортировать стабильной сортировкой (равные элементы сохраняют порядок)

- `sort(It begin, It end)` – отсортировать последовательность
- `stable_sort(It begin, It end)` – отсортировать стабильной сортировкой (равные элементы сохраняют порядок)
- `partial_sort(It begin, It middle, It end)` – сдвинуть младшие `middle - begin` элементов последовательности в начало

Пример реализации алгоритма

```
template <typename InputIt, typename OutputIt>
OutputIt copy(InputIt src_begin, InputIt src_end,
              OutputIt dst_begin)
{
    for (; src_begin != src_end;)
        *dst_begin++ = *src_begin++;

    return dst_begin;
}
```

Пример реализации алгоритма

```
template <typename InputIt, typename OutputIt>
OutputIt copy(InputIt src_begin, InputIt src_end,
              OutputIt dst_begin)
{
    for (; src_begin != src_end;)
        *dst_begin++ = *src_begin++;

    return dst_begin;
}
```

- `*p++ = *q++;` – классический способ скопировать последовательность

Пример реализации алгоритма

```
template <typename InputIt, typename OutputIt,  
          typename UnaryPred>  
OutputIt copy(InputIt src_begin, InputIt src_end,  
              OutputIt dst_begin, UnaryPred pred)  
{  
    for (; src_begin != src_end; ++src_begin)  
        if (pred(*src_begin))  
            *dst_begin++ = *src_begin;  
  
    return dst_begin;  
}
```

Пример реализации алгоритма

```
template <typename InputIt, typename OutputIt,  
          typename UnaryPred>  
OutputIt copy(InputIt src_begin, InputIt src_end,  
              OutputIt dst_begin, UnaryPred pred)  
{  
    for (; src_begin != src_end; ++src_begin)  
        if (pred(*src_begin))  
            *dst_begin++ = *src_begin;  
  
    return dst_begin;  
}
```

- На [cppreference.com](https://en.cppreference.com) почти к любому алгоритму есть примеры реализации

- Хотим реализовать быструю сортировку

```
template <typename It>
void quick_sort(It begin, It end)
{
    // Выбираем опорную точку для сортировки
    const T& pivot = ...;
}
```

- Хотим реализовать быструю сортировку

```
template <typename It>
void quick_sort(It begin, It end)
{
    // Выбираем опорную точку для сортировки
    const T& pivot = ...;
}
```

- Откуда нам взять тип T?

Iterator traits

- Добавим тип элемента в сам класс итератора:

```
template <typename T>
class vector {
    class iterator {
        using value_type = T;
    };
};

template <typename It>
void quick_sort(It begin, It end)
{
    // Выбираем опорную точку для сортировки
    const It::value_type& pivot = ...;
}
```

Iterator traits

- Добавим тип элемента в сам класс итератора:

```
template <typename T>
class vector {
    class iterator {
        using value_type = T;
    };
};

template <typename It>
void quick_sort(It begin, It end)
{
    // Выбираем опорную точку для сортировки
    const It::value_type& pivot = ...;
}
```

- Ошибка компиляции: компилятор не понимает, `It::value_type` это тип или статическая переменная

Iterator traits

- Добавим волшебное слово `typename`:

```
template <typename T>
class vector {
    class iterator {
        using value_type = T;
    };
};

template <typename It>
void quick_sort(It begin, It end)
{
    // Выбираем опорную точку для сортировки
    const typename It::value_type& pivot = ...;
}
```

Iterator traits

- Добавим волшебное слово `typename`:

```
template <typename T>
class vector {
    class iterator {
        using value_type = T;
    };
};

template <typename It>
void quick_sort(It begin, It end)
{
    // Выбираем опорную точку для сортировки
    const typename It::value_type& pivot = ...;
}
```

- Теперь всё компилируется, но не работает если `It` это указатель :(

- Решение: *iterator traits* (от англ. *trait* – свойство, признак, черта)

- Решение: *iterator traits* (от англ. *trait* – свойство, признак, черта)
- Стандартный шаблонный класс, параметризуемый типом итератора, и хранящий некую информацию о нём

Iterator traits

- Решение: *iterator traits* (от англ. *trait* – свойство, признак, черта)
- Стандартный шаблонный класс, параметризуемый типом итератора, и хранящий некую информацию о нём
- По умолчанию достаёт все данные из самого типа итератора:

```
template <typename Iterator>
struct iterator_traits {
    using value_type = typename Iterator::value_type;
    using reference_type = typename Iterator::reference_type;
    ...
};
```

- Есть частичная специализация для указателей:

```
template <typename T>
struct iterator_traits<T*> {
    using value_type = T;
    using reference_type = T&;
    ...
};

template <typename T>
struct iterator_traits<const T*> {
    using value_type = T const;
    using reference_type = T const&;
    ...
};
```

Iterator traits

- Решение нашей проблемы:

```
template <typename It>
void quick_sort(It begin, It end)
{
    // Работает как для указателей, так и для vector::iterator
    using traits = std::iterator_traits<It>;
    const typename traits::value_type& pivot = ...;
}
```

Iterator traits

- Решение нашей проблемы:

```
template <typename It>
void quick_sort(It begin, It end)
{
    // Работает как для указателей, так и для vector::iterator
    using traits = std::iterator_traits<It>;
    const typename traits::value_type& pivot = ...;
}
```

- NB:** во многих случаях вместо `std::iterator_traits` проще использовать вывод типов (`auto`):

```
template <typename It>
void quick_sort(It begin, It end)
{
    const auto& pivot = *begin;
}
```

- `next(it)` – вернуть следующий итератор, не меняя сам `it`

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов
- `advance(it, n)` – сдвинуть `it` (принимается по ссылке) на `n` элементов

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов
- `advance(it, n)` – сдвинуть `it` (принимается по ссылке) на `n` элементов
- `distance(it1, it2)` – на сколько нужно сдвинуть `it1`, чтобы получить `it2`

Алгоритмы на итераторах

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов
- `advance(it, n)` – сдвинуть `it` (принимается по ссылке) на `n` элементов
- `distance(it1, it2)` – на сколько нужно сдвинуть `it1`, чтобы получить `it2`
- Как быстро работают эти алгоритмы?

Алгоритмы на итераторах

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов
- `advance(it, n)` – сдвинуть `it` (принимается по ссылке) на `n` элементов
- `distance(it1, it2)` – на сколько нужно сдвинуть `it1`, чтобы получить `it2`
- Как быстро работают эти алгоритмы? Зависит от конкретного контейнера: $O(1)$ для вектора, $O(N)$ для списка

Алгоритмы на итераторах

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов
- `advance(it, n)` – сдвинуть `it` (принимается по ссылке) на `n` элементов
- `distance(it1, it2)` – на сколько нужно сдвинуть `it1`, чтобы получить `it2`
- Как быстро работают эти алгоритмы? Зависит от конкретного контейнера: $O(1)$ для вектора, $O(N)$ для списка
- Как написать разную реализацию алгоритмов для разных контейнеров?

Алгоритмы на итераторах

- `next(it)` – вернуть следующий итератор, не меняя сам `it`
- `next(it, n)` – вернуть копию, сдвинутую на `n` элементов
- `advance(it, n)` – сдвинуть `it` (принимается по ссылке) на `n` элементов
- `distance(it1, it2)` – на сколько нужно сдвинуть `it1`, чтобы получить `it2`
- Как быстро работают эти алгоритмы? Зависит от конкретного контейнера: $O(1)$ для вектора, $O(N)$ для списка
- Как написать разную реализацию алгоритмов для разных контейнеров? $\implies$ *категории итераторов*

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)
 - *Forward iterator*: только `*it` и `++it`

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)
 - *Forward iterator*: только `*it` и `++it`
 - *Bidirectional iterator*: ещё `--it`

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)
 - *Forward iterator*: только `*it` и `++it`
 - *Bidirectional iterator*: ещё `--it`
 - *Random access iterator*: `it += n` и `it -= n`

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)
 - *Forward iterator*: только `*it` и `++it`
 - *Bidirectional iterator*: ещё `--it`
 - *Random access iterator*: `it += n` и `it -= n`
- Итераторы по вектору или деке – random access

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)
 - *Forward iterator*: только `*it` и `++it`
 - *Bidirectional iterator*: ещё `--it`
 - *Random access iterator*: `it += n` и `it -= n`
- Итераторы по вектору или деке – random access
- Итераторы по списку или set/map – bidirectional

- Категория итератора определяет, какие операции этот итератор умеет выполнять (и эффективно)
 - *Forward iterator*: только `*it` и `++it`
 - *Bidirectional iterator*: ещё `--it`
 - *Random access iterator*: `it += n` и `it -= n`
- Итераторы по вектору или деке – random access
- Итераторы по списку или set/map – bidirectional
- Итераторы по односвязному списку – forward

- В стандартной библиотеке есть специальные классы-теги для определения категорий итераторов:

```
struct forward_iterator_tag {};  
struct bidirectional_iterator_tag : forward_iterator_tag {};  
struct random_access_iterator_tag : bidirectional_iterator_tag{};
```

Категории итераторов

- В стандартной библиотеке есть специальные классы-теги для определения категорий итераторов:

```
struct forward_iterator_tag {};  
struct bidirectional_iterator_tag : forward_iterator_tag {};  
struct random_access_iterator_tag : bidirectional_iterator_tag{};
```

- Они наследуются друг от друга, создавая иерархию категорий

Категории итераторов

- В `iterator_traits` (и в самих классах итераторов) указывается их категория:

```
template <typename T>
class list {
    class iterator {
        using iterator_category = bidirectional_iterator_tag;
    };
};

template <typename T>
struct iterator_traits<T*> {
    using iterator_category = random_access_iterator_tag;
};
```

- Как реализовать `std::next(it, n)` эффективно для разных итераторов?

- Как реализовать `std::next(it, n)` эффективно для разных итераторов? Использовать перегрузку функций:

```
template <typename It>
It next(It it, int n, std::bidirectional_iterator_tag) {
    for (; n > 0; --n)
        ++it;
    return it;
}

template <typename It>
It next(It it, int n, std::random_access_iterator_tag) {
    it += n;
    return it;
}
```

Tag dispatching

```
template <typename It>
It next(It it, int n) {
    using traits = std::iterator_traits<It>;
    return next(it, n, typename traits::iterator_category{});
}
```

Tag dispatching

```
template <typename It>
It next(It it, int n) {
    using traits = std::iterator_traits<It>;
    return next(it, n, typename traits::iterator_category{});
}
```

- Такой подход – перегрузка функции на основе дополнительного параметра-тега – называется *tag dispatching*, и очень распространён в шаблонном коде

- Цикл вида `for (; n > 0; --n)` можно записать смешнее:

```
while (n --> 0) {  
    ...  
}
```

Оператор -->

- Цикл вида `for (; n > 0; --n)` можно записать смешнее:

```
while (n --> 0) {  
    ...  
}
```

- Это не отдельный оператор, а комбинация постфиксного декремента `n--` и сравнения `(n--) > 0`

Оператор -->

- Цикл вида `for (; n > 0; --n)` можно записать смешнее:

```
while (n --> 0) {  
    ...  
}
```

- Это не отдельный оператор, а комбинация постфиксного декремента `n--` и сравнения `(n--) > 0`
- Такой цикл выполнит ровно `n` итераций :)